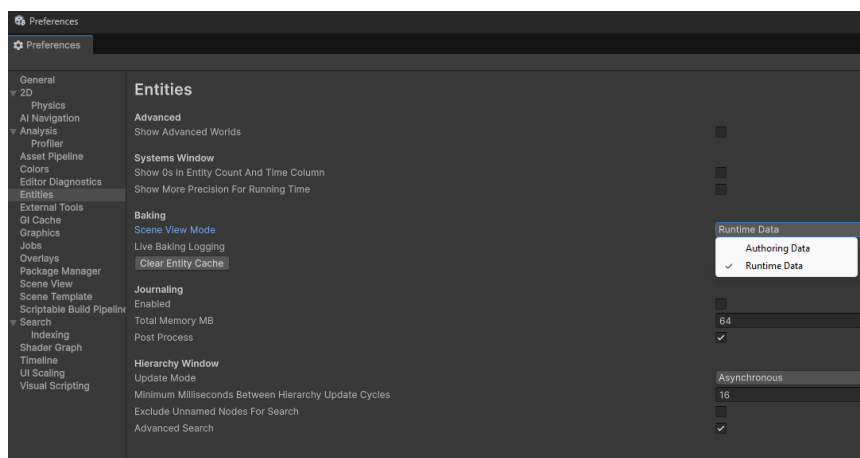


Чтобы Entity двигались визуально не только в окне Game, но и в окне Scene, необходимо изменить в Preferences вот эту настройку на значение 'Runtime Data':



Разберём новые способы хранения данных в DOTS – Blob и DynamicBuffer.

Blob (Binary Large Object) предназначен для хранения статических данных в DOTS. Объект, хранящий данные, располагается в некотором месте в памяти, однако указатель на это место в памяти хранится в компоненте с помощью BlobAssetReference. Blob позволяет хранить вложенные массивы данных, при этом держит их физически рядом в неуправляемой памяти, что даёт большой бонус в плане производительности относительно привычного List<>.

В данном примере в Blob будут записаны волны противников. Структура классов волн, используемых для считывания данных из инспектора, выглядит вот так:

```
[Serializable]
Ссылка: 2
public class EnemiesWave
{
    public float WaitTimeBeforeWave;
    public List<WaveEnemy> EnemiesInWave = new List<WaveEnemy>();
}

[Serializable]
Ссылка: 2
public class WaveEnemy
{
    public EnemyType EnemyType;
    public int Amount;
}

Ссылка: 5
public enum EnemyType : byte
{
    Red, Yellow
}

[SerializeField] List<EnemiesWave> _waves = new List<EnemiesWave>();
```

Перед тем как записывать данные в Blob, для начала нужно прописать структуры, которые будут использоваться. По аналогии с представленными выше классами:

```
Ссылка 4
public struct WavesBlob
{
    public BlobArray<Wave> Waves;
}

Ссылка 2
public struct Wave
{
    public float WaitTimeBeforeWave;
    public BlobArray<EnemyInWave> Enemies;
}

Ссылка 3
public struct EnemyInWave
{
    public EnemyType EnemyType;
    public int Amount;
}
```

Чтобы создать Blob (рекомендуется делать это в Baker скрипте, так как ссылка на Blob будет храниться в компоненте на Entity), необходимо для начала создать вспомогательный объект BlobBuilder, который будет заниматься созданием Blob.

```
Скрипт Unity (1 ссылка на ресурсы) | Ссылка 2
class WavesAuthoring : MonoBehaviour
{
    [SerializeField] List<EnemiesWave> _waves = new List<EnemiesWave>();

    Ссылка 0
    class WavesAuthoringBaker : Baker<WavesAuthoring>
    {
        Ссылка 0
        public override void Bake(WavesAuthoring authoring)
        {
            BlobBuilder blobBuilder = new BlobBuilder(Allocator.Temp);
        }
    }
}
```

Теперь, когда BlobBuilder создан, необходимо создать коренной объект (root). В данном примере это будет эквивалент объекта List<EnemiesWave> _waves.

```
Ссылка 0
public override void Bake(WavesAuthoring authoring)
{
    BlobBuilder blobBuilder = new BlobBuilder(Allocator.Temp);

    ref var root = ref blobBuilder.ConstructRoot<WavesBlob>();
}
```

Теперь, когда root типа WavesBlob создан, для массива BlobArray<Wave> внутри него необходимо выделить память. Длина этого массива фиксированная и указывается при аллокации.

```
Ссылка 0
public override void Bake(WavesAuthoring authoring)
{
    BlobBuilder blobBuilder = new BlobBuilder(Allocator.Temp);

    ref var root = ref blobBuilder.ConstructRoot<WavesBlob>();

    BlobBuilderArray<Wave> wavesBuilderArray = blobBuilder.Allocate(ref root.Waves, authoring._waves.Count);
}
```

Теперь, когда для массива Waves выделена память, его необходимо заполнить значениями типа Wave. Так как структура Wave содержит BlobArray<EnemyInWave> внутри себя, для этого массива тоже необходимо выделить память — внутри каждого из элементов Waves.

```
public override void Bake(WavesAuthoring authoring)
{
    BlobBuilder blobBuilder = new BlobBuilder(Allocator.Temp);

    ref var root = ref blobBuilder.ConstructRoot<WavesBlob>();

    BlobBuilderArray<Wave> wavesBuilderArray = blobBuilder.Allocate(ref root.Waves, authoring._waves.Count);

    for(int i = 0; i < authoring._waves.Count; i++)
    {
        wavesBuilderArray[i].WaitTimeBeforeWave = authoring._waves[i].WaitTimeBeforeWave;

        BlobBuilderArray<EnemyInWave> enemyBuilderArray = blobBuilder.Allocate(ref wavesBuilderArray[i].Enemies, authoring._waves[i].EnemiesInWave.Count);
    }
}
```

Именно так реализуются вложенные массивы данных с помощью Blob. В данном примере всего 1 уровень вложенности, но Blob позволяет делать любой уровень вложенности, главное иметь желание и терпение написать много циклов for друг в друге.

```
public override void Bake(WavesAuthoring authoring)
{
    BlobBuilder blobBuilder = new BlobBuilder(Allocator.Temp);

    ref var root = ref blobBuilder.ConstructRoot<WavesBlob>();

    BlobBuilderArray<Wave> wavesBuilderArray = blobBuilder.Allocate(ref root.Waves, authoring._waves.Count);

    for(int i = 0; i < authoring._waves.Count; i++)
    {
        wavesBuilderArray[i].WaitTimeBeforeWave = authoring._waves[i].WaitTimeBeforeWave;

        BlobBuilderArray<EnemyInWave> enemyBuilderArray = blobBuilder.Allocate(ref wavesBuilderArray[i].Enemies, authoring._waves[i].EnemiesInWave.Count);

        for(int j = 0; j < authoring._waves[i].EnemiesInWave.Count; j++)
        {
            enemyBuilderArray[j] = new EnemyInWave()
            {
                EnemyType = authoring._waves[i].EnemiesInWave[j].EnemyType,
                Amount = authoring._waves[i].EnemiesInWave[j].Amount
            };
        }
    }
}
```

После того, как Blob заполнился данными, можно создать ссылку на него — упомянутый ранее BlobAssetReference. А также на этом этапе BlobBuilder выполнил своё предназначение, а значит, можно освободить память, занимаемую им с помощью Dispose.

```

public override void Bake(WavesAuthoring authoring)
{
    BlobBuilder blobBuilder = new BlobBuilder(Allocator.Temp);
    ref var root = ref blobBuilder.ConstructRoot<WavesBlob>();
    BlobBuilderArray<Wave> wavesBuilderArray = blobBuilder.Allocate(ref root.Waves, authoring._waves.Count);
    for(int i = 0; i < authoring._waves.Count; i++)
    {
        wavesBuilderArray[i].WaitTimeBeforeWave = authoring._waves[i].WaitTimeBeforeWave;
        BlobBuilderArray<EnemyInWave> enemyBuilderArray = blobBuilder.Allocate(ref wavesBuilderArray[i].Enemies, authoring._waves[i].EnemiesInWave.Count);
        for(int j = 0; j < authoring._waves[i].EnemiesInWave.Count; j++)
        {
            enemyBuilderArray[j] = new EnemyInWave()
            {
                EnemyType = authoring._waves[i].EnemiesInWave[j].EnemyType,
                Amount = authoring._waves[i].EnemiesInWave[j].Amount
            };
        }
    }

    BlobAssetReference<WavesBlob> wavesBlobAssetReference = blobBuilder.CreateBlobAssetReference<WavesBlob>(Allocator.Persistent);
    blobBuilder.Dispose();
}

```

Метод AddBlobAsset позволяет нам больше не думать о работе с памятью в контексте этого Blob. Он регистрируется в списке Blob'ов проекта, и память, занятая им, по окончании жизни мира Entity освободится сама, без необходимости вручную вызывать Dispose. Также внутри этого метода происходит дедупликация (если на нескольких объектах висит этот Baker скрипт, то в результате запекания компонента, все Entity с одинаковыми данными, запекаемыми в Blob, получают в BlobAssetReference ссылку на один и тот же Blob, а не создадут несколько идентичных).

```

BlobAssetReference<WavesBlob> wavesBlobAssetReference =
    blobBuilder.CreateBlobAssetReference<WavesBlob>(Allocator.Persistent);

blobBuilder.Dispose();

AddBlobAsset(ref wavesBlobAssetReference, out _);

```

Финальный шаг — добавление созданного BlobAssetReference в компонент Entity.

```

public struct WavesHolder : IComponentData
{
    ...
    public BlobAssetReference<WavesBlob> WavesReference;
}

```

```

BlobAssetReference<WavesBlob> wavesBlobAssetReference =
    blobBuilder.CreateBlobAssetReference<WavesBlob>(Allocator.Persistent);

blobBuilder.Dispose();

AddBlobAsset(ref wavesBlobAssetReference, out _);

Entity entity = GetEntity(TransformUsageFlags.None);

AddComponent(entity, new WavesHolder()
{
    ...
    WavesReference = wavesBlobAssetReference
});

```

Теперь чтобы обратиться к Blob и использовать данные из него внутри системы, достаточно найти компонент с BlobAssetReference. В данном примере это делается через GetSingleton, так как в мире всего 1 Entity с таким компонентом.

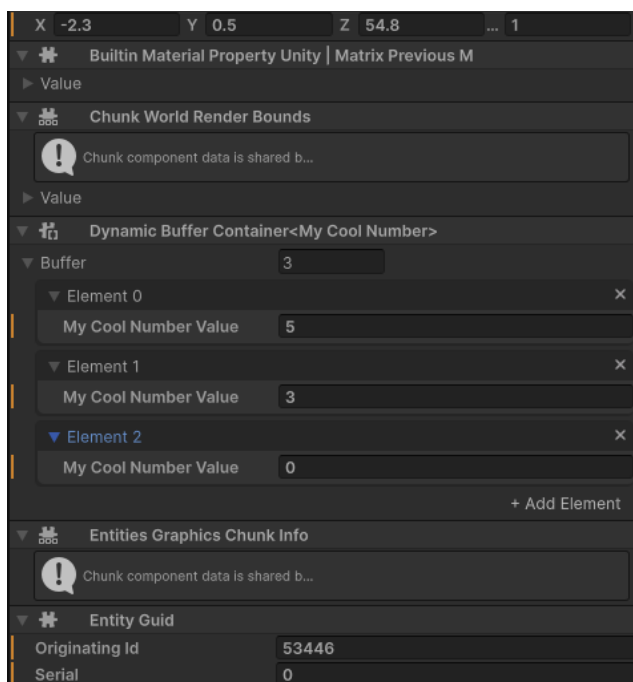
```
[BurstCompile]
Ссылка: 0
public void OnUpdate(ref SystemState state)
{
    ref var waves = ref SystemAPI.GetSingleton<WavesHolder>().WavesReference.Value.Waves;
```

DynamicBuffer — компонент-список. Он хранит динамический список однотипных данных непосредственно на Entity как компонент. Чтобы добавить его на Entity, необходимо для начала создать структуру, реализующую интерфейс IBufferElementData, которая будет отражать данные, хранящиеся в одном элементе этого списка.

```
Ссылка: 0
public struct MyCoolNumber : IBufferElementData
{
    public int MyCoolNumberValue;
}
```

Чтобы добавить DynamicBuffer на Entity, аналогично методу AddComponent в Baker скрипте используется метод AddBuffer.

```
Скрипт Entity System
class MyBufferAuthoring : MonoBehaviour
{
    Ссылка: 0
    class MyBufferAuthoringBaker : Baker<MyBufferAuthoring>
    {
        Ссылка: 0
        public override void Bake(MyBufferAuthoring authoring)
        {
            Entity entity = GetEntity(TransformUsageFlags.Dynamic);
            AddBuffer<MyCoolNumber>(entity);
        }
    }
}
```



У `DynamicBuffer` есть понятие вместимости – максимального количества элементов. По умолчанию это значение будет равно такому количеству элементов, которое вместится в 128 байт, однако поменять это значение на необходимое можно с помощью атрибута `InternalBufferCapacity`. Превышение максимального количества элементов в процессе работы игры не приведёт к ошибке, однако негативно скажется на производительности, поэтому максимальную необходимую в проекте вместимость для каждого `DynamicBuffer` лучше продумать сразу.

```
[InternalBufferCapacity(16)]
Ссылка: 1
public struct MyCoolNumber : IBufferElementData
{
    public int MyCoolNumberValue;
}
```

Как было написано выше, `DynamicBuffer` – по сути своей, компонент-список, а следовательно, как и другие компоненты, он влияет на архетип `Entity` и может являться фильтром для систем. Чтобы использовать `DynamicBuffer` в системе в цикле `foreach`, ему не нужны обёртки `RefRO` или `RefRW`. Достаточно просто перечислить его с другими компонентами.

```
partial struct MyTestSystem : ISystem
{
    [BurstCompile]
    Ссылка: 0
    public void OnUpdate(ref SystemState state)
    {
        foreach(var (buffer, movement) in SystemAPI.Query<DynamicBuffer<MyCoolNumber>, RefRW<Movement>>())
        {
            buffer.Add(new MyCoolNumber()
            {
                MyCoolNumberValue = (int)movement.ValueRO.Speed
            });
        }
    }
}
```

Как и для обычных компонентов, для `DynamicBuffer` есть свои версии `GetSingleton` и `ComponentLookup`.

```
[BurstCompile]
Ссылка: 0
public void OnUpdate(ref SystemState state)
{
    DynamicBuffer<MyCoolNumber> myBuffer = SystemAPI.GetSingletonBuffer<MyCoolNumber>(false);
    BufferLookup<MyCoolNumber> bufferLookup = SystemAPI.GetBufferLookup<MyCoolNumber>(false);
}
```

Для любого действия, связанного с перестройкой чанков памяти (создание и удаление `Entity`, изменение архетипа `Entity` посредством добавления и удаления компонентов) в большинстве сценариев следует

использовать EntityCommandBuffer. Что логично из названия, это буфер команд. Необходимые команды записываются в этот буфер, когда цикл проходимся по Entity, а затем по завершении этого цикла, когда перестройка чанков памяти безопасна, EntityCommandBuffer воспроизводит все эти команды разом в том порядке, в котором они были в него записаны.

```
[BurstCompile]
Ссылка 0
public void OnUpdate(ref SystemState state)
{
    EntityCommandBuffer ecb = new EntityCommandBuffer(Allocator.Temp); // создание EntityCommandBuffer

    foreach(var (health, entity) in SystemAPI.Query<RefRO<Health>>().WithEntityAccess())
    {
        if(health.ValueRO.CurrentHealth <= 0)
        {
            ecb.Instantiate(corpsse); // запись в EntityCommandBuffer команды создания Entity
            ecb.DestroyEntity(entity); // запись в EntityCommandBuffer команды удаления Entity
        }
    }

    // Уже после того, как система прошла по всем необходимым Entity, происходит выполнение всех записанных команд
    ecb.Playback(state.EntityManager);

    ecb.Dispose(); // Освобождение памяти. EntityCommandBuffer – одноразовый инструмент, после Playback он больше не нужен
}
```

Чтобы получить «случайные» значения в DOTS используется структура Unity.Mathematics.Random. Значения, которые эта структура возвращает при использовании её методов, чётко детерминированы сидом, задаваемым при создании структуры. По этой причине имеет смысл генерировать для Unity.Mathematics.Random случайный сид при запекании с помощью UnityEngine.Random или использовать id Entity как сид (в ситуации, когда сид нужно сгенерировать для объекта, находящегося на сцене в момент запуска плеймода, подойдёт UnityEngine.Random, однако он перестанет делать то, что от него ожидается, когда речь пойдёт о спавне Entity в рантайме. Для таких Entity лучше всё же использовать id Entity).

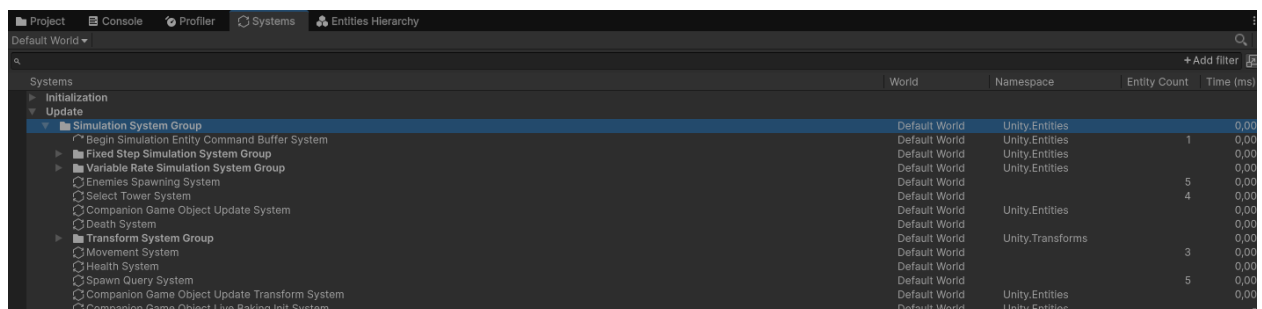
```
class SpawnpointsAuthoringBaker : Baker<SpawnpointsAuthoring>
{
    Ссылка 0
    public override void Bake(SpawnpointsAuthoring authoring)
    {
        Entity entity = GetEntity(TransformUsageFlags.None);
        AddComponent(entity, new RandomHolder()
        {
            rng = new Unity.Mathematics.Random()
            {
                state = (uint)UnityEngine.Random.Range(1, 1000000)
            }
        });
        // или
        AddComponent(entity, new RandomHolder()
        {
            rng = new Unity.Mathematics.Random()
            {
                state = (uint)entity.Index
            }
        });
    }
}
```

Хранение префабов Entity, которые будет необходимо спавнить в рантайме, оптимальнее всего переложить в один Singleton-Component, чтобы хотя и немного, но экономить ресурсы оперативной памяти. Чтобы обратиться к такому «держателю префабов» из любой системы, достаточно использовать метод SystemAPI.GetSingleton.

```
class PrefabHolderAuthoring : MonoBehaviour
{
    [SerializeField] GameObject _redEnemyPrefab;
    [SerializeField] GameObject _yellowEnemyPrefab;
    Ссылка: 0
    class PrefabHolderAuthoringBaker : Baker<PrefabHolderAuthoring>
    {
        Ссылка: 0
        public override void Bake(PrefabHolderAuthoring authoring)
        {
            Entity entity = GetEntity(TransformUsageFlags.None);

            AddComponent(entity, new PrefabsHolder()
            {
                RedEnemyPrefab = GetEntity(authoring._redEnemyPrefab, TransformUsageFlags.Dynamic),
                YellowEnemyPrefab = GetEntity(authoring._yellowEnemyPrefab, TransformUsageFlags.Dynamic)
            });
        }
    }
}
Ссылка: 6
public struct PrefabsHolder : IComponentData
{
    public Entity RedEnemyPrefab;
    public Entity YellowEnemyPrefab;
}
```

Во вкладке просмотра систем несложно заметить, что все системы находятся в разных группах, которые визуальнo отображаются, как папки. По умолчанию все создаваемые системы появляются в SimulationSystemGroup – группе, которая обновляется 1 раз в кадр, как обычный метод Update из MonoBehaviour.



Systems	World	Namespace	Entity Count	Time (ms)
Initialization				
Update				
Simulation System Group	Default World	Unity.Entities		0,00
Begin Simulation Entity Command Buffer System	Default World	Unity.Entities	1	0,00
Fixed Step Simulation System Group	Default World	Unity.Entities		0,00
Variable Rate Simulation System Group	Default World	Unity.Entities		0,00
Enemies Spawning System	Default World		5	0,00
Select Tower System	Default World		4	0,00
Companion Game Object Update System	Default World	Unity.Entities		0,00
Death System	Default World			0,00
Transform System Group	Default World	Unity.Transforms		0,00
Movement System	Default World		3	0,00
Health System	Default World			0,00
Spawn Query System	Default World		5	0,00
Companion Game Object Update Transform System	Default World	Unity.Entities		0,00
Companion Game Object Live Baking Init System	Default World	Unity.Entities		0,00

Однако если вдруг возникла необходимость поменять группу, в которой система обновляется, это делается с помощью атрибута UpdateInGroup. Как и пройденные в прошлый раз UpdateBefore и UpdateAfter, этот атрибут пишется в квадратных скобках над системой.


```

using Unity.Burst;
using Unity.Entities;

[UpdateInGroup(typeof(FixedStepSimulationSystemGroup))]
Ссылка: 1
public partial struct SpawnQuerySystem : ISystem
{
    [BurstCompile]
    Ссылка: 0
    public void OnUpdate(ref SystemState state) ...
}

```

Также существуют сценарии, в которых может понадобиться собственная группа систем. Например, для нескольких систем сразу нужна своя частота обновления – раз в 3 или 5 секунд. Или 10 раз в кадр. Возможно, это группа систем, необходимая для инициализации, которая сработает 1 раз и затем будет выключена. Вне зависимости от того, зачем такое может понадобиться, делается это следующим образом:

```

using Unity.Entities;

[UpdateInGroup(typeof(SimulationSystemGroup))]
Ссылка: 0
public partial class RegenerationSystemGroup : ComponentSystemGroup
{
    Ссылка: 0
    protected override void OnUpdate()
    {
        base.OnUpdate();
    }
}

```

Эта группа систем «RegenerationSystemGroup» обновляется внутри группы SimulationSystemGroup и на данный момент, как и родительская группа, обновляется в каждом кадре. Чтобы понять, как управлять частотой выполнения систем в группе, достаточно знать, что системы внутри группы выполняются тогда, когда вызывается `base.OnUpdate()`. На основе этой информации легко написать, например, группу систем, которая будет срабатывать ровно 1 раз в секунду:

```

[UpdateInGroup(typeof(SimulationSystemGroup))]
Ссылка: 2
public partial class RegenerationSystemGroup : ComponentSystemGroup
{
    float timer = 0;
    Ссылка: 0
    protected override void OnUpdate()
    {
        timer += SystemAPI.Time.DeltaTime;

        if (timer >= 1f)
        {
            base.OnUpdate();
            timer = timer - 1f;
        }
    }
}

```

Теперь чтобы добавить в эту группу систему, достаточно дописать ей уже знакомый атрибут `UpdateInGroup` с нужной группой. Группы можно делать вложенными без ограничений, если существует такая необходимость.

```
using Unity.Burst;|
using Unity.Entities;
using Unity.Mathematics;

[UpdateInGroup(typeof(RegenerationSystemGroup))]
Ссылка: 1
partial struct RegenerationSystem : ISystem
{
    [BurstCompile]
    Ссылка: 0
    public void OnUpdate(ref SystemState state)...
```

